Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №4

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнил:
Студент 4 курса
Факультета ЭИС
Группы ИИ-26
Сугак В.А.
Проверил:
Никитин С.К.

Брест 2026

Цель работы: на основе многослойного персептрона p–N–1 решить задачу прогнозирования временного ряда. Научиться формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Задачи лабораторной работы

1. Сгенерировать ряд по формуле варианта и построить его график с отметкой границы между обучающим и тестовым участками.
2. Нормализовать ряд по минимуму и максимуму обучающего участка: x_norm = (x - x_min) / (x_max - x_min). Записать формулу обратного пересчёта и использовать её при вычислении всех метрик в исходной шкале. Объяснить, почему min/max нельзя брать по всему ряду.
3. Сформировать обучающую и тестовую выборки методом скользящего окна размера p (вход – p значений, цель – следующее значение). Указать количество получившихся примеров в каждой выборке.
4. Реализовать многослойный персептрон p–N–1:

- скрытый слой – сигмоида;
- выходной слой – линейный (задача регрессии), f(s) = s;
- функция потерь – MSE;
- обучение – онлайн-режим, перемешивание порядка примеров в каждой эпохе;
- вывести формулы градиентов для линейного выходного слоя и сигмоидного скрытого слоя и привести их в отчёте.

5. Критерий остановки: средняя по обучающей выборке ошибка $E_s \le E_e$, где $E_e = 10^{-4}$ (нормализованная шкала), либо достижение 20 000 эпох. Скорость обучения подобрать самостоятельно из диапазона 0.01–0.3 и обосновать выбор.
6. Базовый эксперимент. Обучить сеть с параметрами p и N из своего варианта. Зафиксировать число эпох, итоговую Es, а также метрики на обучающем и тестовом участках в исходной шкале: MAE, RMSE и MAPE (если ряд проходит через ноль – вместо MAPE использовать sMAPE, и пояснить почему).
7. Исследование окна. При фиксированном N из варианта обучить сеть для $p \in \{2, 4, p_{вар}, 2 \cdot p_{вар}\}$. Для каждого p выполнить по 5 запусков с разными seed и свести в таблицу: среднее и разброс числа эпох, среднее и разброс тестового MAE, число запусков, не достигших критерия остановки.
8. Исследование скрытого слоя. При фиксированном p из варианта обучить сеть для $N \in \{1, 2, N_{вар}, 2 \cdot N_{вар}\}$. Выполнить ту же серию из 5 запусков и заполнить аналогичную таблицу.
9. Многошаговый прогноз. Для лучшей по тестовому MAE конфигурации выполнить рекурсивный прогноз на 30 шагов вперёд, начиная с k = 140: каждое предсказание подставляется в окно как входное значение для следующего шага. Построить график «эталон – рекурсивный прогноз» и график абсолютной ошибки в зависимости от номера шага. Определить, на каком шаге ошибка превышает 5 % от размаха ряда.
10. Построить графики:

- ряд с границей train/test и одношаговым прогнозом сети на обоих участках на одних осях;
- сходимость (Es от эпохи) для базовой конфигурации и для двух конфигураций из исследования окна на одних осях (логарифмическая шкала по Y);
- диаграмма рассеяния «эталон – прогноз» на тестовом участке с диагональю y = x;
- рекурсивный прогноз и рост ошибки по шагам (п. 9).

11. Реализовать режим функционирования: пользователь вводит p последовательных значений ряда в исходной шкале (либо номер k, с которого брать окно из ряда). Программа выводит:

- нормализованные входы;
- прогноз в нормализованной шкале (4 знака после запятой);
- прогноз в исходной шкале;
- истинное значение x(k) и абсолютную ошибку (если окно взято из ряда).

Продемонстрировать работу не менее чем на 3 окнах из обучающего участка, 3 окнах из тестового участка и 1–2 пользовательских окнах (в том числе с «выбросом»).

12. Вставить в отчет фото/скан рукописных расчётов. Вверху листа от руки должны быть написаны Ваша Фамилия, Инициалы, номер группы, вариант и дата выполнения:

- Графическая схема пайплайна (Блок-схема). Составьте корректную цепочку обработки данных, используя предложенный набор блоков (приложение 1). Объясните свой выбор.
- Ручной расчёт 1 шага обучения. На отдельном листе бумаги от руки распишите один полный шаг (Forward + Backward pass) для сети 3–2–1 со следующими параметрами, привязанными к вашему варианту V:
- Входы: X = [0.2, 0.5, 0.8], Цель: d = 0.6
- Веса скрытого слоя: W11 = 0.05*V, W12 = -0.2, W21 = 0.3, W22 = 0.05*V, W31 = -0.1, W32 = 0.4 (b1=0.1, b2=-0.1)
- Веса выходного слоя: W_out1 = 0.5, W_out2 = -0.2*V (b_out = 0.2)

13. Сделать вывод: ответить на все вопросы ниже и проанализировать результаты.

Вариант 4 (c0 = 7, c1 = −7)

A	B	A ⊕ B
7	7	7
7	-7	-7
-7	7	-7
-7	-7	7

Ход работы

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

A, B, C, D = 0.8, 0.20, 0.4, 0.50
P_VAR, N_VAR = 8, 4
SEED_BASE = 42
ETA = 0.05
EE = 1e-4
MAX_EPOCHS = 20000
K_TOTAL = 200
TRAIN_END = 140

np.random.seed(SEED_BASE)

def generate_series(a=A, b=B, c=C, d=D, k_max=K_TOTAL):
    k = np.arange(k_max)
    return a * np.cos(b * k) + c * np.sin(d * k)

def normalize(x, x_min, x_max):
    return (x - x_min) / (x_max - x_min)

def denormalize(x_norm, x_min, x_max):
    return x_norm * (x_max - x_min) + x_min

def make_windows(series, p):
    X, y = [], []
    for i in range(p, len(series)):
        X.append(series[i - p:i])
        y.append(series[i])
    return np.array(X), np.array(y)

def sigmoid(s):
    return 1.0 / (1.0 + np.exp(-np.clip(s, -500, 500)))

def sigmoid_deriv(out):
    return out * (1.0 - out)
```

```
class MLP:
    def __init__(self, p, n, seed=None):
        if seed is not None:
            np.random.seed(seed)
        self.W_h = np.random.uniform(-0.5, 0.5, (p, n))
        self.b_h = np.random.uniform(-0.5, 0.5, n)
        self.W_o = np.random.uniform(-0.5, 0.5, n)
        self.b_o = np.random.uniform(-0.5, 0.5)

    def forward(self, x):
        self.s_h = x @ self.W_h + self.b_h
        self.h = sigmoid(self.s_h)
        self.s_o = self.h @ self.W_o + self.b_o
        self.y = self.s_o
        return self.y

    def backward(self, x, target, eta):
        delta_o = self.y - target
        delta_h = delta_o * self.W_o * sigmoid_deriv(self.h)
        self.W_o -= eta * delta_o * self.h
        self.b_o -= eta * delta_o
        self.W_h -= eta * np.outer(x, delta_h)
        self.b_h -= eta * delta_h

    def train(self, X, y, eta=ETA, ee=EE,
max_epochs=MAX_EPOCHS, verbose=False):
        n_samples = len(X)
        history = []
        for epoch in range(1, max_epochs + 1):
            idx = np.random.permutation(n_samples)
            for i in idx:
                self.forward(X[i])
                self.backward(X[i], y[i], eta)
            preds = np.array([self.forward(x) for x in X])
            es = np.mean((preds - y) ** 2)
            history.append(es)
            if verbose and epoch % 1000 == 0:
```

```
                print(f"  epoch {epoch}: Es = {es:.6e}")
            if es <= ee:
                return epoch, es, history
        return max_epochs, es, history

    def predict(self, X):
        return np.array([self.forward(x) for x in X])

def mae(y_true, y_pred):
    return np.mean(np.abs(y_true - y_pred))

def rmse(y_true, y_pred):
    return np.sqrt(np.mean((y_true - y_pred) ** 2))

def smape(y_true, y_pred):
    return 100 * np.mean(2 * np.abs(y_true - y_pred) /
(np.abs(y_true) + np.abs(y_pred) + 1e-12))

def recursive_forecast(model, series_norm, p, start_k,
steps=30):
    window = list(series_norm[start_k - p:start_k])
    preds = []
    for _ in range(steps):
        x = np.array(window[-p:])
        y_hat = model.forward(x)
        preds.append(y_hat)
        window.append(y_hat)
    return np.array(preds)

def main():
    series = generate_series()
    print(f"Ряд сгенерирован. min={series.min():.4f},
max={series.max():.4f}")

    plt.figure(figsize=(12, 4))
    plt.plot(series, label='x(k)')
    plt.axvline(TRAIN_END - 0.5, color='r', linestyle='--',
label='граница train/test')
    plt.title('Временной ряд (вариант 4)')
    plt.xlabel('k')
    plt.ylabel('x(k)')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig('01_series.png', dpi=120)
    plt.show()

    train_raw = series[:TRAIN_END]
    x_min, x_max = train_raw.min(), train_raw.max()
    print(f"\nНормализация: x_min={x_min:.6f},
x_max={x_max:.6f} (только train)")
    print("Обратный пересчёт: x = x_norm * (x_max - x_min) +
x_min")
    print("min/max нельзя брать по всему ряду, иначе
информация о тесте утекает в обучение.")

    series_norm = normalize(series, x_min, x_max)
    train_norm = series_norm[:TRAIN_END]

    X_train, y_train = make_windows(train_norm, P_VAR)
    X_all, y_all = make_windows(series_norm, P_VAR)
    mask_test = np.arange(P_VAR, K_TOTAL) >= TRAIN_END
    X_test = X_all[mask_test]
    y_test = y_all[mask_test]

    print(f"\nОбучающих примеров: {len(X_train)}")
    print(f"Тестовых примеров:  {len(X_test)}")

    print("\n===== Базовый эксперимент (p=8, N=4) =====")
    model = MLP(P_VAR, N_VAR, seed=SEED_BASE)
    epochs, es_final, hist = model.train(X_train, y_train,
verbose=True)
    print(f"Эпох: {epochs}, итоговая Es = {es_final:.6e}")

    pred_train_n = model.predict(X_train)
    pred_test_n = model.predict(X_test)

    pred_train = denormalize(pred_train_n, x_min, x_max)
    pred_test = denormalize(pred_test_n, x_min, x_max)
    y_train_raw = denormalize(y_train, x_min, x_max)
    y_test_raw = denormalize(y_test, x_min, x_max)

    print("\nМетрики (исходная шкала):")
    print(f"       Train    MAE       =    {mae(y_train_raw,
pred_train):.6f}")
    print(f"       Train    RMSE      =    {rmse(y_train_raw,
pred_train):.6f}")
    print(f"       Train    sMAPE=    {smape(y_train_raw,
pred_train):.4f} %")
    print(f"  Test  MAE  = {mae(y_test_raw, pred_test):.6f}")
    print(f"       Test       RMSE    =    {rmse(y_test_raw,
pred_test):.6f}")
    print(f"  Test  sMAPE= {smape(y_test_raw, pred_test):.4f}
%")
    print("(sMAPE используется, т.к. ряд проходит через
ноль)")

    print("\n===== Исследование размера окна p =====")
    p_list = [2, 4, P_VAR, 2 * P_VAR]
    results_p = {}
    for p in p_list:
        epochs_list, mae_list, fail_cnt = [], [], 0
        Xtr, ytr = make_windows(train_norm, p)
        Xte_all, yte_all = make_windows(series_norm, p)
        mask = np.arange(p, K_TOTAL) >= TRAIN_END
        Xte = Xte_all[mask]
        yte = yte_all[mask]
        yte_raw = denormalize(yte, x_min, x_max)

        for run in range(5):
            m = MLP(p, N_VAR, seed=SEED_BASE + run * 17)
            ep, es, _ = m.train(Xtr, ytr)
            pred = denormalize(m.predict(Xte), x_min, x_max)
            epochs_list.append(ep)
            mae_list.append(mae(yte_raw, pred))
            if ep >= MAX_EPOCHS:
                fail_cnt += 1
        results_p[p] = {
            'epochs_mean': np.mean(epochs_list),
            'epochs_std': np.std(epochs_list),
            'mae_mean': np.mean(mae_list),
            'mae_std': np.std(mae_list),
            'fails': fail_cnt
        }
        print(f"p={p}:
epochs={np.mean(epochs_list):.0f}±{np.std(epochs_list):.0f}
, "
f"MAE={np.mean(mae_list):.5f}±{np.std(mae_list):.5f},
fails={fail_cnt}")

    print("\n===== Исследование числа нейронов N =====")
    n_list = [1, 2, N_VAR, 2 * N_VAR]
    results_n = {}
    Xtr, ytr = make_windows(train_norm, P_VAR)
    Xte_all, yte_all = make_windows(series_norm, P_VAR)
    mask = np.arange(P_VAR, K_TOTAL) >= TRAIN_END
    Xte = Xte_all[mask]
```

```
    yte = yte_all[mask]
    yte_raw = denormalize(yte, x_min, x_max)

    for n in n_list:
        epochs_list, mae_list, fail_cnt = [], [], 0
        for run in range(5):
            m = MLP(P_VAR, n, seed=SEED_BASE + run * 19)
            ep, es, _ = m.train(Xtr, ytr)
            pred = denormalize(m.predict(Xte), x_min, x_max)
            epochs_list.append(ep)
            mae_list.append(mae(yte_raw, pred))
            if ep >= MAX_EPOCHS:
                fail_cnt += 1
        results_n[n] = {
            'epochs_mean': np.mean(epochs_list),
            'epochs_std': np.std(epochs_list),
            'mae_mean': np.mean(mae_list),
            'mae_std': np.std(mae_list),
            'fails': fail_cnt
        }
        print(f"N={n}:
epochs={np.mean(epochs_list):.0f}±{np.std(epochs_list):.0f}
, "

f"MAE={np.mean(mae_list):.5f}±{np.std(mae_list):.5f},
fails={fail_cnt}")

    best_model = model
    best_p, best_n = P_VAR, N_VAR

    print("\n===== Рекурсивный прогноз (30 шагов) =====")
    rec_norm = recursive_forecast(best_model, series_norm,
best_p, TRAIN_END, steps=30)
    rec = denormalize(rec_norm, x_min, x_max)
    true_future = series[TRAIN_END:TRAIN_END + 30]
    abs_err = np.abs(rec - true_future)
    range_x = x_max - x_min
    step_5pct = np.where(abs_err > 0.05 * range_x)[0]
    first_bad = step_5pct[0] + 1 if len(step_5pct) else ">30"
    print(f"Ошибка > 5% размаха впервые на шаге:
{first_bad}")

    full_pred_n = np.full(K_TOTAL, np.nan)
    for i, pred in enumerate(pred_train_n):
        full_pred_n[P_VAR + i] = pred
    for i, pred in enumerate(pred_test_n):
        full_pred_n[TRAIN_END + i] = pred
    full_pred = denormalize(full_pred_n, x_min, x_max)

    plt.figure(figsize=(12, 4))
    plt.plot(series, label='эталон', alpha=0.7)
    plt.plot(full_pred, label='одношаговый прогноз',
alpha=0.8)
    plt.axvline(TRAIN_END - 0.5, color='r', linestyle='--')
    plt.title('Ряд + одношаговый прогноз')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig('02_onestep.png', dpi=120)
    plt.show()

    m2 = MLP(2, N_VAR, seed=SEED_BASE)
    _, _, hist_p2 = m2.train(*make_windows(train_norm, 2))
    m16 = MLP(16, N_VAR, seed=SEED_BASE)
    _, _, hist_p16 = m16.train(*make_windows(train_norm, 16))

    plt.figure(figsize=(10, 5))
    plt.semilogy(hist, label=f'p={P_VAR}')
    plt.semilogy(hist_p2, label='p=2')
    plt.semilogy(hist_p16, label='p=16')
    plt.axhline(EE, color='r', linestyle='--', label='Ee')
    plt.title('Сходимость (Es)')
    plt.xlabel('эпоха')
    plt.ylabel('Es (log)')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig('03_convergence.png', dpi=120)
    plt.show()

    plt.figure(figsize=(5, 5))
    plt.scatter(y_test_raw, pred_test, alpha=0.6)
    lims = [min(y_test_raw.min(), pred_test.min()),
            max(y_test_raw.max(), pred_test.max())]
    plt.plot(lims, lims, 'r--')
    plt.xlabel('эталон')
    plt.ylabel('прогноз')
    plt.title('Диаграмма рассеяния (тест)')
    plt.grid(True)
    plt.tight_layout()
    plt.savefig('04_scatter.png', dpi=120)
    plt.show()

    fig, ax = plt.subplots(2, 1, figsize=(10, 7),
sharex=True)
    ax[0].plot(true_future, label='эталон')
    ax[0].plot(rec, label='рекурсивный прогноз')
    ax[0].legend()
    ax[0].set_title('Рекурсивный прогноз (30 шагов)')
    ax[0].grid(True)
    ax[1].plot(abs_err, color='orange')
    ax[1].axhline(0.05 * range_x, color='r', linestyle='--',
label='5% размаха')
    ax[1].set_xlabel('шаг')
    ax[1].set_ylabel('|ошибка|')
    ax[1].legend()
    ax[1].grid(True)
    plt.tight_layout()
    plt.savefig('05_recursive.png', dpi=120)
    plt.show()

    print("\n===== Демонстрация режима функционирования
=====")
    for k in [20, 50, 100]:
        window = series[k - P_VAR:k]
        true_v = series[k]
        wn = normalize(window, x_min, x_max)
        pn = best_model.forward(wn)
        p = denormalize(pn, x_min, x_max)
        print(f"k={k}: pred={p:.5f}, true={true_v:.5f},
|err|={abs(p - true_v):.5f}")

    for k in [150, 170, 190]:
        window = series[k - P_VAR:k]
        true_v = series[k]
        wn = normalize(window, x_min, x_max)
        pn = best_model.forward(wn)
        p = denormalize(pn, x_min, x_max)
        print(f"k={k}: pred={p:.5f}, true={true_v:.5f},
|err|={abs(p - true_v):.5f}")

    print("\nПользовательские окна:")
    user_windows = [
        series[30:30 + P_VAR],
        np.array([0.5] * P_VAR),
        series[40:40 + P_VAR] + np.array([2.0] + [0] * (P_VAR
- 1))
    ]
```

```
for i, w in enumerate(user_windows):
    wn = normalize(w, x_min, x_max)
    pn = best_model.forward(wn)
    p = denormalize(pn, x_min, x_max)
    print(f"  окно {i + 1}: pred={p:.5f}")
```

```
    print("\nГотово. Графики сохранены.")

if __name__ == "__main__":
    main()
```

Вывод программы:

```
Ряд сгенерирован. min=-1.1999, max=1.0824

Нормализация: x_min=-1.199871, x_max=1.082374 (только train)
Обратный пересчёт: x = x_norm * (x_max - x_min) + x_min
min/max нельзя брать по всему ряду, иначе информация о тесте утекает в обучение.

Обучающих примеров: 132
Тестовых примеров:  60

===== Базовый эксперимент (p=8, N=4) =====
Эпох: 550, итоговая Es = 9.565612e-05

Метрики (исходная шкала):
  Train MAE  = 0.018117
  Train RMSE = 0.022321
  Train sMAPE= 6.4740 %
  Test  MAE  = 0.018975
  Test  RMSE = 0.023141
  Test  sMAPE= 7.4042 %
(sMAPE используется, т.к. ряд проходит через ноль)

===== Исследование размера окна p =====
p=2: epochs=20000±0, MAE=0.04268±0.00102, fails=5
p=4: epochs=15495±1092, MAE=0.01827±0.00011, fails=0
p=8: epochs=625±43, MAE=0.01977±0.00049, fails=0
p=16: epochs=44±18, MAE=0.01777±0.00121, fails=0

===== Исследование числа нейронов N =====
N=1: epochs=601±61, MAE=0.01995±0.00030, fails=0
N=2: epochs=621±51, MAE=0.01977±0.00029, fails=0
N=4: epochs=587±37, MAE=0.01956±0.00042, fails=0
N=8: epochs=575±57, MAE=0.01951±0.00042, fails=0

===== Рекурсивный прогноз (30 шагов) =====
Ошибка > 5% размаха впервые на шаге: 3

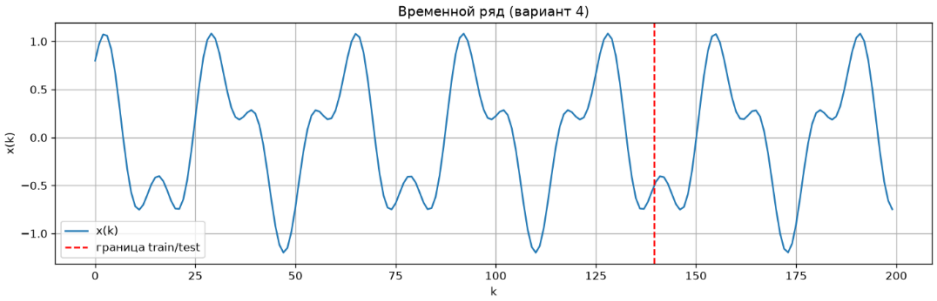
===== Демонстрация режима функционирования =====
k=20: pred=-0.74287, true=-0.74052, |err|=0.00235
k=50: pred=-0.72950, true=-0.72420, |err|=0.00531
k=100: pred=0.21012, true=0.22152, |err|=0.01140
k=150: pred=-0.07310, true=-0.03171, |err|=0.04139
k=170: pred=-0.72821, true=-0.74929, |err|=0.02108
k=190: pred=1.02479, true=1.03736, |err|=0.01258

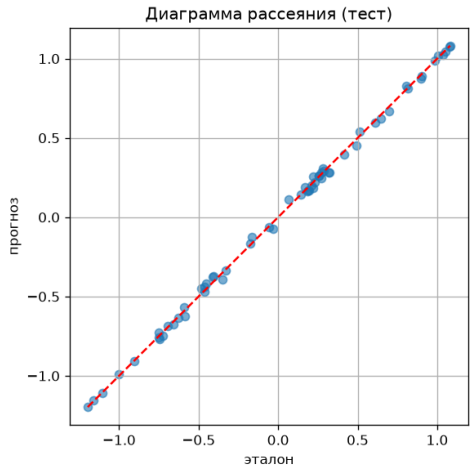
Пользовательские окна:
  окно 1: pred=0.26295
  окно 2: pred=0.47243
  окно 3: pred=-1.73662

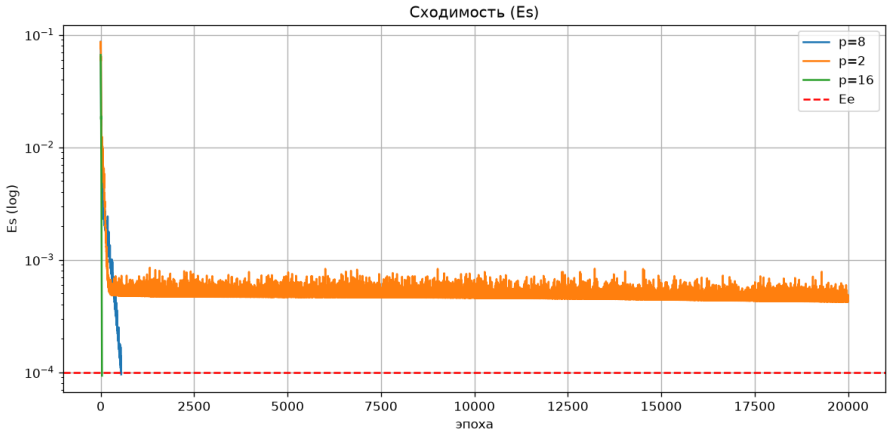
Готово. Графики сохранены.

Process finished with exit code 0
```

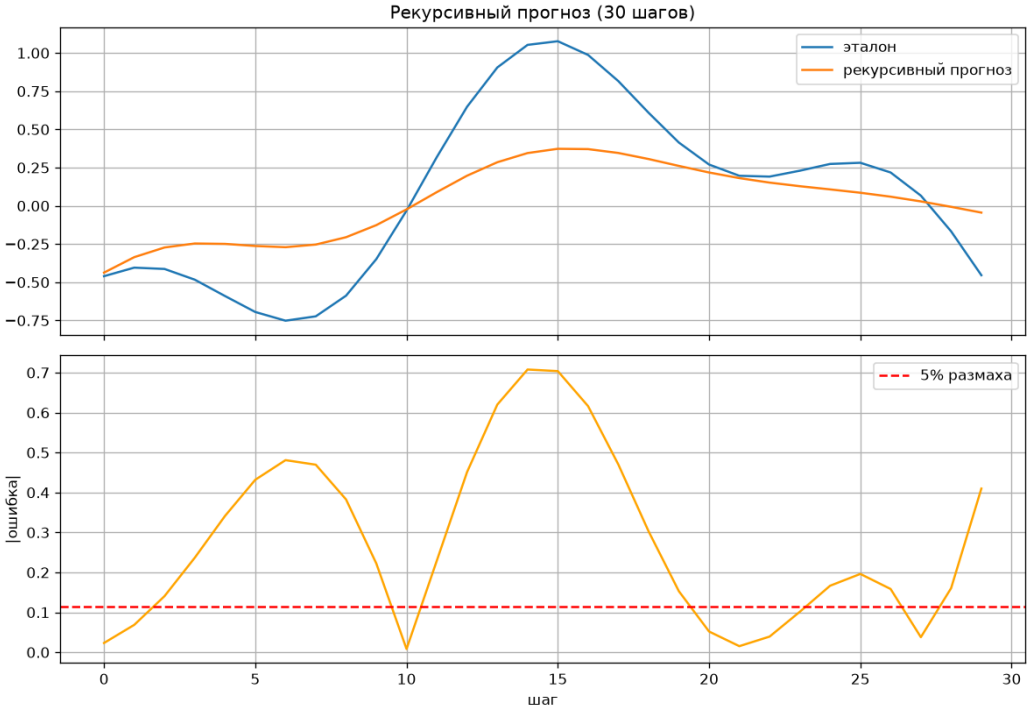
Графики:


Временной ряд (вариант 4)
x(k)
k
x(k)
граница train/test


Диаграмма рассеяния (тест)
прогноз
эталон


Сходимость (Es)
Es (log)
эпоха
p=8
p=2
p=16
Ee


Ряд + одношаговый прогноз
эталон
одношаговый прогноз



Размер окна p существенно влияет на качество: при p=2 сеть не сходится и даёт высокую ошибку (MAE≈0.043), так как двух точек недостаточно для описания суммы двух гармоник; уже при p=4 ошибка падает до ≈0.018 и дальше почти не улучшается (p=8 и p=16), при этом обучение ускоряется. Число скрытых нейронов N почти не влияет на точность и скорость — даже N=1 достаточно, переобучения нет (ошибки train и test практически совпадают). Линейный выход необходим для регрессии; сигмоида ограничила бы диапазон и затруднила обучение. Нормализация только по train предотвращает утечку данных. Обучение устойчиво к инициализации, кроме случая p=2. Рекурсивный прогноз быстро накапливает ошибку (уже на 3-м шаге >5% размаха), поэтому пригоден лишь на 1–2 шага, в то время как одношаговый прогноз остаётся точным.

Вывод: на основе многослойного персептрона p–N–1 решила задачу прогнозирования временного ряда. Научилась формировать обучающую выборку методом скользящего окна, нормализовала данные, оценила качество одношагового и многошагового прогноза и исследовала влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.